

UNIVERSIDAD CENTRAL DEL ECUADOR

Facultad de Ingeniería y Ciencias Aplicadas

Carrera de Computación

TRABAJO GRUPAL

INTEGRANTES:

- KEVIN ALBUJA
- FERNANDEZ PAMELA
- OSORIO DIEGO

Semestre:
6to Semestre

Fecha de entrega:
18/07/2025

Docente:
Ingeniero Jaime Salvador

Inversión de Control (IoC)

La Inversión de Control (IoC) es un principio de diseño que consiste en ceder a un contenedor o framework la responsabilidad de gestionar el ciclo de vida de los objetos y sus dependencias. Esto permite un código más modular y desacoplado. En vez de que una clase cree sus propias dependencias, estas le son inyectadas desde el exterior.

Este principio es la base para el desarrollo de software orientado a componentes, donde las clases se enfocan exclusivamente en su lógica de negocio y delegan la gestión de sus colaboraciones a un contenedor externo. Esto permite una mayor escalabilidad, mantenimiento más sencillo y facilita la aplicación de patrones como el de inversión de dependencias.

Inyección de Dependencias (DI)

La Inyección de Dependencia o “DI”, es una técnica concreta de aplicar IoC. Consiste en inyectar las dependencias de una clase desde el exterior por medio de un contenedor de IoC. Esto reduce el acoplamiento y facilita la reutilización y pruebas.

Podemos decir que el DI en lugar de que una clase instancie directamente sus colaboraciones internas, las recibe mediante su constructor, métodos setters o directamente como atributos. Esta técnica es clave en frameworks modernos como Spring o CDI.

Tipos de DI:

- **Constructor**: Las dependencias se inyectan al momento de construir el objeto, garantizando que el objeto esté completamente inicializado.
- **Setter**: Las dependencias se establecen después de la construcción, permitiendo cierta flexibilidad en la configuración.
- **Por campo**: Las dependencias se inyectan directamente en los atributos, generalmente usando anotaciones.

Ejemplo:

```
public class Motor {
	private final Combustible combustible;

	public Motor(Combustible combustible) {
		this.combustible = combustible;
	}
}
```

Ventajas:

- Desacoplamiento
- Facilidad de pruebas
- Mayor modularidad
- Reutilización
- Claridad en las dependencias de cada clase

ORM (Object-Relational Mapping)

ORM es una técnica para mapear clases a tablas de base de datos relacionales. JPA es la especificación Java estándar para ORM, y Hibernate es su implementación más conocida.

- El uso de ORM permite que los desarrolladores trabajen a nivel de objetos en vez de tener que escribir manualmente sentencias SQL, ayudando a la productividad y reduciendo errores.
- Facilita la portabilidad entre distintas bases de datos y permite aprovechar características avanzadas como la caché de segundo nivel, transacciones, y relaciones complejas entre entidades.

CDI + Weld 6.2.8 (Jakarta CDI 4)

El CDI o Contexts and Dependency Injection permite la inyección de dependencias y la gestión de contexto como su nombre lo indica, su objetivo principal es promover el desarrollo de aplicaciones altamente cohesivas y débilmente acopladas.

Weld, en su versión 6.2.8, es la implementación de referencia de esta especificación y puede ejecutarse de forma independiente en aplicaciones Java SE. Usar CDI con Weld permite aplicar IoC y DI sin necesidad de Spring.

Anotaciones en CDI + Weld

- **@Dependent:** Indica que el bean tiene un ciclo de vida dependiente del objeto que lo inyecta.
- **@ApplicationScoped:** Define un bean que vive durante toda la vida de la aplicación. Es útil para servicios que deben compartirse.
- **@Inject:** Marca una dependencia que debe ser inyectada automáticamente por el contenedor (similar a @Autowired en Spring).

La gran ventaja de CDI es que permite declarar componentes (beans) con anotaciones que indican su ciclo de vida, y también permite establecer las dependencias entre ellos mediante el @Inject.

Ejemplo:

```
@Dependent
public class ServicioSaludo {
    public void saludar() {
        System.out.println("Hola desde CDI 4 y Weld 6.2.8!");
    }
}

@ApplicationScoped
public class Aplicacion {
    @Inject
    ServicioSaludo servicio;

    public void iniciar() {
        servicio.saludar();
    }
}
```

Main (standalone con Weld SE):

```
public class Main {
    public static void main(String[] args) {
        try (SeContainer container =
SeContainerInitializer.newInstance().initialize()) {
            Aplicacion app = container.select(Aplicacion.class).get();
            app.iniciar();
        }
    }
}
```

Dependencia Maven:

```
<dependency>
  <groupId>org.jboss.weld.se</groupId>
  <artifactId>weld-se-core</artifactId>
  <version>6.2.8.Final</version>
</dependency>
```

Spring Framework

Spring Framework se utiliza para desarrollar aplicaciones empresariales en Java, donde el Spring posee un amplio conjunto de herramientas como acceso a datos, seguridad y mensajería. Dentro del framework, los módulos Core y Data Access son fundamentales.

Anotaciones en Spring Framework

- **@Configuration:** Declara una clase como fuente de definiciones de beans. Equivale a un archivo de configuración.
- **@Bean:** Indica que el método produce un bean que debe ser registrado en el contenedor Spring.
- **@Repository:** Marca una clase como componente de acceso a datos y habilita la captura automática de excepciones relacionadas a la persistencia.

Spring Core + Data Access

El módulo Core contiene el contenedor de IoC, que permite definir y gestionar beans, aplicar inyección de dependencias mediante anotaciones como @Bean, y utilizar el contexto de aplicación (ApplicationContext) como punto de entrada del ciclo de vida del sistema.

Por su parte, el módulo Data Access incluye herramientas como JdbcTemplate, que simplifican el trabajo con bases de datos relacionales.

Configuración Java:

```java
@Configuration
public class AppConfig {
    @Bean
    public ServicioSaludo saludo() {
        return new ServicioSaludo();
    }

    @Bean
    public Aplicacion app() {
        return new Aplicacion(saludo());
    }
}
```

Uso de ApplicationContext:

```
AnnotationConfigApplicationContext context = new
AnnotationConfigApplicationContext(AppConfig.class);
Aplicacion app = context.getBean(Aplicacion.class);
app.iniciar();
```

Spring JDBC

Spring JDBC es parte del módulo Data Access de Spring y permite interactuar con bases de datos relacionales usando SQL, pero eliminando el código repetitivo.

El Spring JDBC posee clases como JdbcTemplate o NamedParameterJdbcTemplate, las cuales ayudan a realizar consultas y operaciones sobre la base de datos de forma sencilla, manteniendo el control total sobre las sentencias SQL.

DAO con JdbcTemplate:

```
@Repository
public class ClienteDao {
    private final JdbcTemplate jdbcTemplate;

    public ClienteDao(JdbcTemplate jdbcTemplate) {
        this.jdbcTemplate = jdbcTemplate;
    }

    public List<Cliente> obtenerTodos() {
        return jdbcTemplate.query("SELECT * FROM clientes",
            (rs, rowNum) -> new Cliente(rs.getLong("id"), rs.getString("nombre")));
    }
}
```

Configuración:

```
@Configuration
public class JdbcConfig {
    @Bean
    public DataSource dataSource() {
        return new DriverManagerDataSource("jdbc:h2:mem:testdb", "sa", "");
    }

    @Bean
    public JdbcTemplate jdbcTemplate(DataSource dataSource) {
```

```
        return new JdbcTemplate(dataSource);
    }
}
```

JPA + Hibernate (sin Spring Boot)

Java Persistence API (JPA) ayuda al mapeo objeto-relacional (ORM) en Java. Define un conjunto de anotaciones y APIs para interactuar con bases de datos de manera orientada a objetos. Hibernate es una de las implementaciones más robustas y populares de JPA.

Al usar JPA con Hibernate, el programador solo se preocupa por crear sus clases y relaciones, y deja que el propio framework se encargue de guardar y recuperar esos objetos en la base de datos. De esta manera se trabaja más rápido y el código queda más limpio y fácil de entender.

Anotaciones en JPA (Hibernate)

- **@Entity:** Indica que una clase es una entidad JPA, y será persistida en la base de datos.
- **@Table(name = "..."):** Asocia la clase con una tabla específica de la base de datos.
- **@Id:** Indica el atributo que actúa como clave primaria.
- **@GeneratedValue:** Define que el valor del ID será generado automáticamente (ej. auto-incremental).

Entidad:

```
@Entity
@Table(name = "clientes")
public class Cliente {
    @Id
    @GeneratedValue
    private Long id;
    private String nombre;
}
```

Persistencia:

```
EntityManagerFactory emf = Persistence.createEntityManagerFactory("miUnidad");
EntityManager em = emf.createEntityManager();
Cliente c = new Cliente();
c.setNombre("Ana");
em.getTransaction().begin();
em.persist(c);
em.getTransaction().commit();
```

Spring Data JDBC 3.5.1

Spring Data JDBC es una opción más sencilla y ligera que JPA cuando se trabaja con Spring. En lugar de usar cosas complejas como el “contexto de persistencia” o proxies automáticos, aquí se escribe las sentencias SQL y el framework simplemente las ejecuta. Funciona bien con el estilo de diseño de DDD (Domain-Driven Design), porque organiza el código y deja el control del SQL al programador.

Anotaciones en Spring Data JDBC

- **@Table("..."):** Especifica el nombre de la tabla asociada con la clase (entidad).
- **@Id:** Marca el campo como clave primaria para Spring Data JDBC.
- **@EnableJdbcRepositories:** Habilita el uso de repositorios JDBC declarativos en el contexto de Spring.
- **@Bean:** También se utiliza aquí para registrar manualmente los componentes necesarios (como DataSource, JdbcTemplate, etc.).

Entidad:

```
@Table("clientes")
public class Cliente {
    @Id
    private Long id;
    private String nombre;
}
```

Repositorio:

```
public interface ClienteRepository extends CrudRepository<Cliente, Long> {
    List<Cliente> findByNombre(String nombre);
}
```

Configuración sin Boot:

```
@Configuration
@EnableJdbcRepositories
public class Config extends AbstractJdbcConfiguration {

    @Bean
    public DataSource dataSource() {
        return new DriverManagerDataSource("jdbc:h2:mem:testdb", "sa", "");
```

```
    }

    @Bean
    public NamedParameterJdbcOperations operations(DataSource ds) {
        return new NamedParameterJdbcTemplate(ds);
    }
}
```

Tecnologías para Inversión de Control y Acceso a Datos

Tecnología	Nivel de Abstracción	Contenedor IoC	ORM Integrado	SQL Manual	Uso principal
CDI + Weld	Medio	Sí (Weld)	No	Opcional	Aplicaciones Jakarta EE
Spring Framework Core	Medio-Alto	Sí	Opcional	Sí	Contenedor IoC + Configuración
Spring Data Access	Bajo	Sí	No	Sí	Acceso directo con JDBC
JPA + Hibernate	Alto	No (usa JPA)	Sí	No	ORM completo
Spring Data JDBC	Medio	Sí	Limitado	Sí	Alternativa a JPA + SQL limpio

CONCLUSIÓN

- Los principios de Inversión de Control (IoC) y la Inyección de Dependencias (DI) no solo son principio teóricos en la arquitectura moderna de software, sino que son directamente aplicables mediante herramientas robustas como CDI, Spring Framework, JPA/Hibernate y Spring Data JDBC.

- IoC permite entregar la creación y gestión de objetos a un contenedor, ayudando en el desacoplamiento y facilitando la extensibilidad de las aplicaciones. DI, por su parte, habilita una arquitectura donde las dependencias son entregadas explícitamente desde el exterior, lo que mejora la claridad, la mantenibilidad y la capacidad de prueba del código.

- CDI y Weld proporcionan una solución estándar, ligera y portable para la inyección de dependencias en el ecosistema Jakarta EE.

- Spring Framework extiende este concepto con un contenedor IoC que además de ofrecer acceso con mayor facilidad a los datos mediante su módulo Core y Data Access.
- JPA y Hibernate permiten trabajar directamente con clases y objetos de Java, donde, el framework se encarga de convertir todo eso en operaciones en la base de datos. Así, se evita escribir sentencias SQL, mientras Hibernate hace el trabajo de guardarlos y recuperarlos en las tablas.
- Spring Data JDBC proporciona una alternativa más simple y controlada al ORM, para sistemas que requieren simplicidad, transparencia y control total sobre el SQL.

REFERENCIAS BIBLIOGRÁFICAS:

- Cosmina, I., Harrop, R., & Schaefer, C. (2023). *Pro Spring 6*. Apress.
- Deinum, M., & Rubio, D. (2023). *Spring 6 Recipes: A Problem-Solution Approach to Spring Framework* (5th ed.). Apress.
- Ottinger, J. B. (2022). *Beginning Spring: From Beginner to Pro* (2nd ed.). Apress.
- Gutierrez, F., & Ottinger, J. B. (2022). *Introducing Spring: Learning and Building Java-based Applications With Spring* (2nd ed.). Apress.